

The Final Rip

MIRAGE 2.0 — Reverse Engineering / Anti-Debugging — Walkthrough

CATEGORY	PORT	FILES
Reverse Engineering / Anti-Debug	1666/tcp	final_rip.c (source provided)
VULNERABILITY	STATUS	
N/A — validator reversal, not memory corruption	Solved	

Step 1 — First Contact

A menu-driven “core link” terminal: Inspect Core, Stabilize Core, Attempt Final Rip, Core Status, Disconnect. Trying “Attempt Final Rip” immediately prints “The Core is not stable enough” — so something has to happen first. “Inspect Core” hints at it directly: “EXTRACTION LOCK: ENGAGED” and “DIAGNOSTIC NOTE: 'HE DOES NOT WAIT.'” — a nudge that whatever the solve is, it can't be slow (fitting, since source is provided and the natural temptation with a CTF pwn-styled menu is to reach for a debugger).

Step 2 — Reading the Source Instead of Attaching a Debugger

Since full source is given, the smarter first move is reading it rather than immediately attaching gdb — and reading it explains why: two functions are clearly defensive.

- `debugger_present()` reads `/proc/self/status` and checks whether `TracerPid` is non-zero — the standard way a process detects that something has `ptrace`'d it, which is exactly what happens the moment a debugger attaches.
- `timing_trip()` times a 500,000-iteration loop and fails if it took over 250ms — catching single-stepping or breakpoints, which slow execution down dramatically even without a debugger attaching via `ptrace` in the traditional sense.

Both checks live directly inside `final_rip()`, and either one tripping sets `stone_state = -1` and ends the connection. That confirms the instinct from Step 1: this has to be solved by connecting normally and reasoning about the logic offline, not by attaching a debugger to watch it happen live.

Step 3 — Finding What Actually Needs to Be True

Working backward from `final_rip()`, the real success condition (beyond passing the two anti-debug checks) is `stone_state == 42 && rip_lock == 0x5A17`. `rip_lock` is never modified anywhere in the source, so it's always true by default — it's decoration, not an obstacle. `stone_state` is only ever set to 42 inside `stabilize()`, and only when the value typed in matches the output of a small mixing function applied to a fixed constant:

```
static unsigned int hidden_validator(unsigned int x){
    x ^= 0xA17C39E1u;
    x = x * 0x45D9F3Bu + 0x1337u;
    x ^= x >> 16;
    x *= 0x27D4EB2Du;
    x ^= x >> 13;
    return x;
}
unsigned int target = hidden_validator(0xC0DEFACEu);
```

Since every input and every operation here is a fixed compile-time constant, the “correct” stabilization vector can simply be computed once, offline, in Python — there’s no need to brute force or interact with the live process to find it at all:

```
>>> def hidden_validator(x):
...     x &= 0xFFFFFFFF
...     x ^= 0xA17C39E1
...     x = (x * 0x45D9F3B + 0x1337) & 0xFFFFFFFF
...     x ^= x >> 16
...     x = (x * 0x27D4EB2D) & 0xFFFFFFFF
...     x ^= x >> 13
...     return x
...
>>> hidden_validator(0xC0DEFACE)
4220478953
```

Step 4 — Running It

```
$ nc <host> 1666
THANOS@CORE> 2
[CORE] Enter stabilization vector: 4220478953
[CORE] Phase lock restored. The Core resists the extraction.
THANOS@CORE> 3
```

Sending option 2 with the computed vector sets `stone_state = 42`. Sending option 3 immediately afterward — from a plain nc connection with no debugger attached — passes both anti-debug checks and both state checks, and `final_rip()` XOR-0x5A-decodes and prints the flag.

FLAG

```
MIRAGE{th3_final_rip_w4s_st0pp3d_1n_t1m3!}
```

Takeaway

When a challenge visibly defends against debuggers, that’s a strong signal the intended solution doesn’t need one — the answer is meant to be derivable offline from the logic alone. Fighting the anti-debug measures head-on is almost always more work than simply reading what they’re protecting.